

Animacare: Project's System Design, Algorithms, Dfd And Table Design

Part A: System Design

1. Architecture Overview

AnimaCare uses a **3-Tier Decoupled Client-Server Architecture**:

Tier	Technology	Role
Presentation	React.js 18+ (Vite)	SPA Frontend, Role-based dashboards
Application	Django 4+ / Django REST Framework	RESTful API, Business Logic, JWT Auth
Data	PostgreSQL (AWS RDS) / SQLite (Dev)	Relational persistence
Storage	AWS S3	Pet images, X-Rays, Prescriptions

2. Technology Stack

Frontend:

- React.js with Vite (fast HMR builds)
- React Router DOM (protected role-based routing)
- React Context API (AuthContext for global state)
- Vanilla CSS (module-scoped stylesheets)

Backend:

- Python 3.10+ / Django 4+

- Django REST Framework (serializers, viewsets)
- Custom JWT Authentication (CustomJWTAuthentication using PyJWT + HS256)
- Django ORM (model-driven schema)

Database:

- **Production:** PostgreSQL on AWS RDS — ACID-compliant, concurrent connections, automated backups
- **Development:** SQLite3 — Zero-config, file-based for fast local iteration

Cloud:

- AWS S3 — Media file storage (pet photos, diagnostic media)
- AWS RDS — Managed PostgreSQL hosting

3. Module Architecture (RBAC)

The backend is split into 7 Django apps, each serving a specific role:

Module	Django App	User Role	Purpose
1. Authentication	apps.users	All Users	Registration, Login, JWT tokens, Profiles
2. Pet Management	apps.citizens	Citizen	Pet registration, Pet Passport, ownership
3. SOS & Rescue	apps.citizens	Citizen + Civic	SOS alerts, shelter assignment
4. Adoption Portal	apps.shelter	Citizen + Shelter Admin	Browse animals, apply, review applications
5. Clinical Records	apps.clinical	Veterinarian	Consultations, vaccines, prescriptions, media

Module	Django App	User Role	Purpose
6. Analytics & AI	apps.analytics	All Roles	Compatibility scoring, health risk flagging
7. Governance	apps.governance	Super Admin	User approvals, audit trails, system health

4. API Routing Architecture

/api/auth/ → apps.users.urls (Login, Register, Profile)

/api/citizens/ → apps.citizens.urls (Pets, SOS Alerts)

/api/shelter/ → apps.shelter.urls (Inventory, Adoptions)

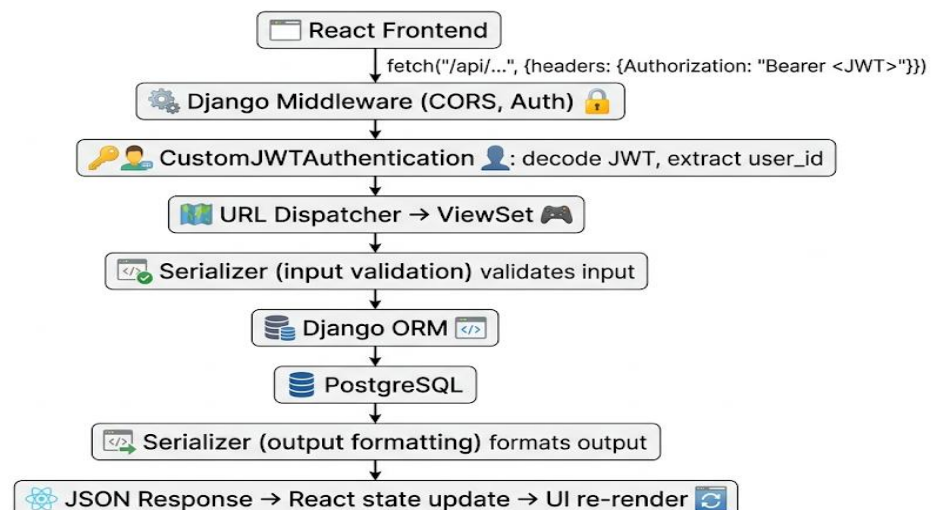
/api/clinical/ → apps.clinical.urls (Consultations, Vaccines, Rx)

/api/v4/ → apps.analytics.urls (AI Scoring, Health Dashboard)

/api/superadmin/ → apps.governance.urls (User Mgmt, Audit Logs)

/api/public-health/ → apps.public_health.urls (Outbreak Analytics)

5. Request Lifecycle



6. Authentication Flow

Registration:

User submits form → Validate fields → Hash password → Save to Users DB

→ If role is citizen: `account_status = 'pending'` (auto or active)

→ If role is vet/shelter/civic: `account_status = 'pending'` → Needs SuperAdmin approval

Login:

User submits email + password → Validate credentials against DB

→ If valid and `account_status == 'active'`: Generate JWT (HS256, 30min expiry)

→ Return `{access_token, refresh_token, user role}`

→ Frontend stores token in `AuthContext` → Redirects to role-based dashboard

7. Security Architecture

Layer	Mechanism
Authentication	JWT Bearer tokens (PyJWT, HS256 signing)
Authorization	Role check in each View (<code>request.user.role</code> , <code>is_superuser</code>)
Data Integrity	SHA-256 hash on frozen vaccination and prescription records
Audit Trail	Immutable <code>AuditTrail</code> table logs all admin actions with IP
Media Security	AWS S3 pre-signed URLs for file access

Part B: Algorithms

Algorithm 1: Role-Based Access Control (RBAC)

Module: All API Endpoints (Middleware) **Purpose:** Prevent unauthorized access to role-restricted endpoints.

Pseudocode:

ALGORITHM: RBAC_Verify

INPUT: request (Authorization header, endpoint)

OUTPUT: ALLOW or DENY

BEGIN

token = request.headers["Authorization"].split("Bearer ")[1]

TRY

decoded = JWT.decode(token, SECRET_KEY, "HS256")

CATCH ExpiredError, DecodeError

RETURN DENY (401: "Invalid or expired token")

END TRY

user = DB.get(User, id=decoded.user_id)

role = user.role

PERMISSION_MATRIX = {

"citizen": [Pet_CRUD, SOS_Create, Adoption_Apply],

"veterinarian": [Clinical_CRUD, Prescription_Create, Media_Upload],

"shelter_admin": [Inventory_CRUD, Application_Review],

```
"civic_authority": [Analytics_View, Rescue_Dispatch],  
  
"admin":      [ALL_ACCESS]  }  
  
IF request.endpoint IN PERMISSION_MATRIX[role] THEN  
  
    RETURN ALLOW  
  
ELSE  
  
    RETURN DENY (403: "Forbidden")  
  
END IF  
  
END
```

Complexity: Time $O(n)$ where n = permissions per role, Space $O(1)$

Algorithm 2: Pet Duplicate Detection

Module: Citizens (apps.citizens) **Purpose:** Prevent accidental re-registration of the same pet by the same owner.

Pseudocode:

ALGORITHM: Register_Pet

INPUT: owner_id, pet_data (name, species, breed, dob, photo)

OUTPUT: new_pet_record OR duplicate_warning

BEGIN

IF pet_data.name IS EMPTY OR pet_data.species IS EMPTY THEN

RETURN Error(400, "Name and Species required")

END IF

existing = DB.query(

```
"SELECT * FROM Pet WHERE owner_id = ?  
  
AND LOWER(name) = LOWER(?) AND species = ?",  
  
owner_id, pet_data.name, pet_data.species  
  
)  
  
IF existing IS NOT EMPTY THEN  
  
    RETURN Warning("Similar pet already registered")  
  
END IF  
  
IF pet_data.photo EXISTS THEN  
  
    image_url = S3.upload(pet_data.photo)  
  
END IF  
  
new_pet = DB.insert("Pet", {  
  
    owner_id, name, species, breed, dob, media_url: image_url  
  
})  
  
RETURN new_pet  
  
END
```

Complexity: Time $O(n)$ where n = owner's pets, Space $O(1)$

Algorithm 3: Predictive Health Risk Flagging

Module: Analytics (apps.analytics) **Purpose:** Detect overweight/underweight risks by comparing a pet's current weight against breed-specific baselines.

Pseudocode:

ALGORITHM: Health_Risk_Analysis

INPUT: all_pets (list of Pet records)

OUTPUT: flags_created (integer count)

BEGIN

flags_created = 0

BREED_BASELINES = {

"Golden Retriever": {ideal: 30.0 kg, variance: 15%},

"Labrador": {ideal: 32.0 kg, variance: 15%},

"Pug": {ideal: 8.0 kg, variance: 10%}

}

FOR EACH pet IN all_pets DO

latest_record = pet.medical_records.latest("date")

current_weight = latest_record.weight_kg OR pet.current_weight_kg

baseline = BREED_BASELINES.get(pet.breed)

IF baseline IS NULL THEN CONTINUE

upper_limit = baseline.ideal * (1 + baseline.variance)

lower_limit = baseline.ideal * (1 - baseline.variance)

IF current_weight > upper_limit THEN

severity = "High" IF current_weight > upper_limit * 1.1 ELSE "Medium"

DB.insert("HealthRiskFlag", {

pet, risk_type: "Overweight", severity,


```
        description: "Weight exceeds safe range for breed"

    })

    flags_created += 1

ELSE IF current_weight < lower_limit THEN

    DB.insert("HealthRiskFlag", {

        pet, risk_type: "Underweight", severity: "Medium",

        description: "Weight below safe threshold for breed"

    })

    flags_created += 1

END IF

END FOR

RETURN flags_created

END
```

Complexity: Time $O(n)$ where n = total pets, Space $O(1)$

Algorithm 4: SHA-256 Record Immutability (Vaccination & Prescription)

Module: Clinical (apps.clinical) **Purpose:** Ensure clinical vaccination and prescription records cannot be tampered with after finalization by generating a cryptographic hash.

Pseudocode:

ALGORITHM: Freeze_Clinical_Record

INPUT: record (VaccinationLog or DigitalPrescription)

OUTPUT: record with record_hash set

BEGIN

IF record.is_frozen == TRUE AND record.id IS NULL (new record) THEN

 // Build a deterministic data string

 IF record IS VaccinationLog THEN

 data_string = record.vaccine_name + "-" +

 record.batch_number + "-" +

 record.date_administered

 ELSE IF record IS DigitalPrescription THEN

 data_string = STRING(record.medications) +

 STRING(record.issued_date)

 END IF

 // Generate SHA-256 hash

 record.record_hash = SHA256(data_string.encode("utf-8")).hexdigest()

END IF

```
DB.save(record)

RETURN record

END

VERIFICATION ALGORITHM:

To verify integrity later:

recompute_hash = SHA256(same_data_string)

IF recompute_hash == record.record_hash THEN

    RETURN "Record is AUTHENTIC"

ELSE

    RETURN "Record has been TAMPERED"

END IF
```

Complexity: Time $O(1)$, Space $O(1)$

Complexity: Time $O(1)$, Space $O(1)$

Algorithm 5: Adoption Application State Machine

Module: Shelter (apps.shelter) **Purpose:** Manage the lifecycle of an adoption application through defined states.

Pseudocode:

ALGORITHM: Process_Adoption_Application

INPUT: application_id, new_status, admin_user

OUTPUT: updated_application

BEGIN

```
app = DB.get("AdoptionApplication", id=application_id)

// Define valid state transitions

VALID_TRANSITIONS = {

    "Pending"    → ["Under Review", "Cancelled"],

    "Under Review" → ["Interview Scheduled", "Approved", "Rejected"],

    "Interview Scheduled" → ["Approved", "Rejected"],

    "Approved"    → [], // Terminal state

    "Rejected"    → [], // Terminal state

    "Cancelled"   → []  // Terminal state

}

IF new_status NOT IN VALID_TRANSITIONS[app.status] THEN

    RETURN Error(400, "Invalid state transition")

END IF

app.status = new_status

DB.save(app)

// If approved, update animal status

IF new_status == "Approved" THEN

    animal = app.animal

    animal.is_adopted = TRUE

    animal.is_available = FALSE
```

```
DB.save(animal)

END IF

// Log to audit trail

Log_Audit_Trail(admin_user, "APPLICATION_" + new_status, app)

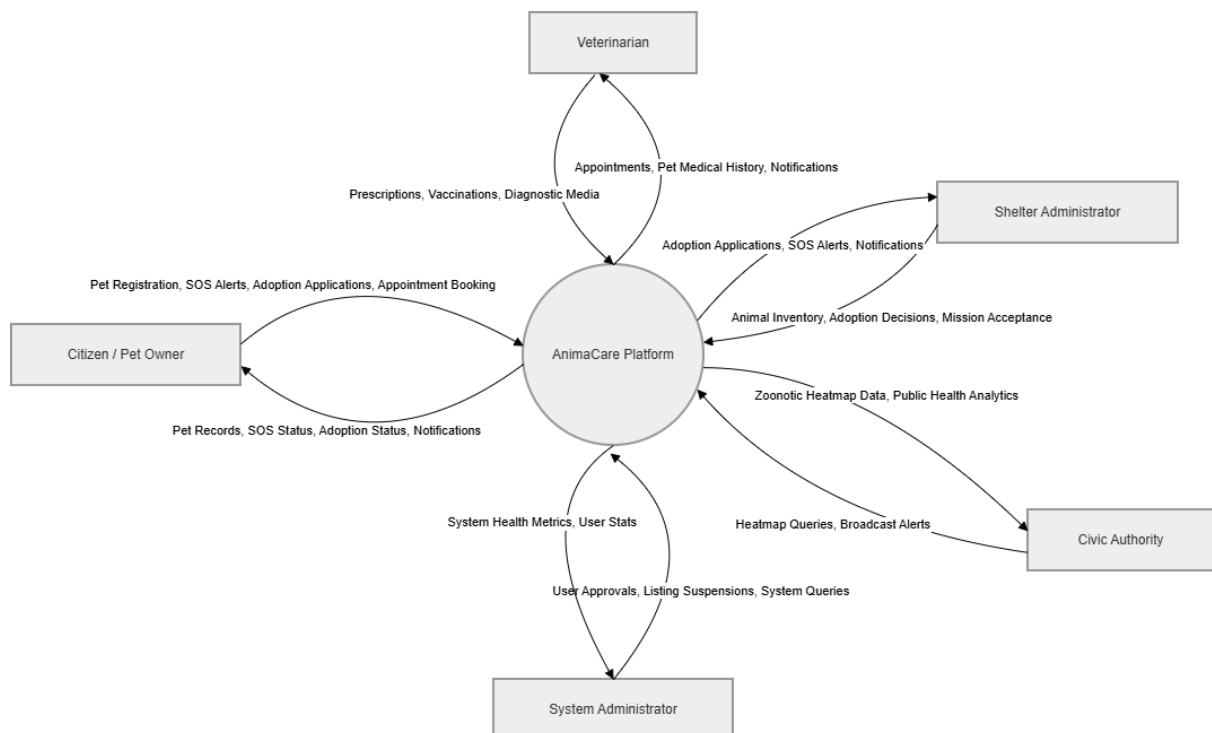
RETURN app

END
```

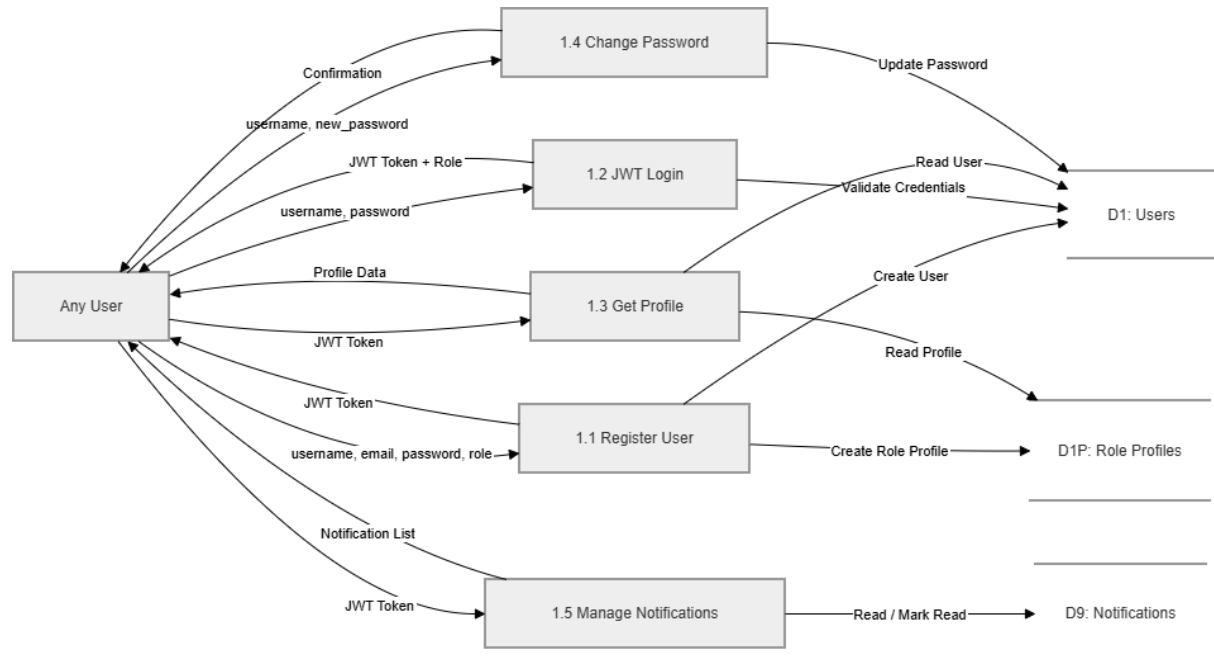
Complexity: Time $O(1)$, Space $O(1)$

Part C: DFD (Data Flow Diagram)

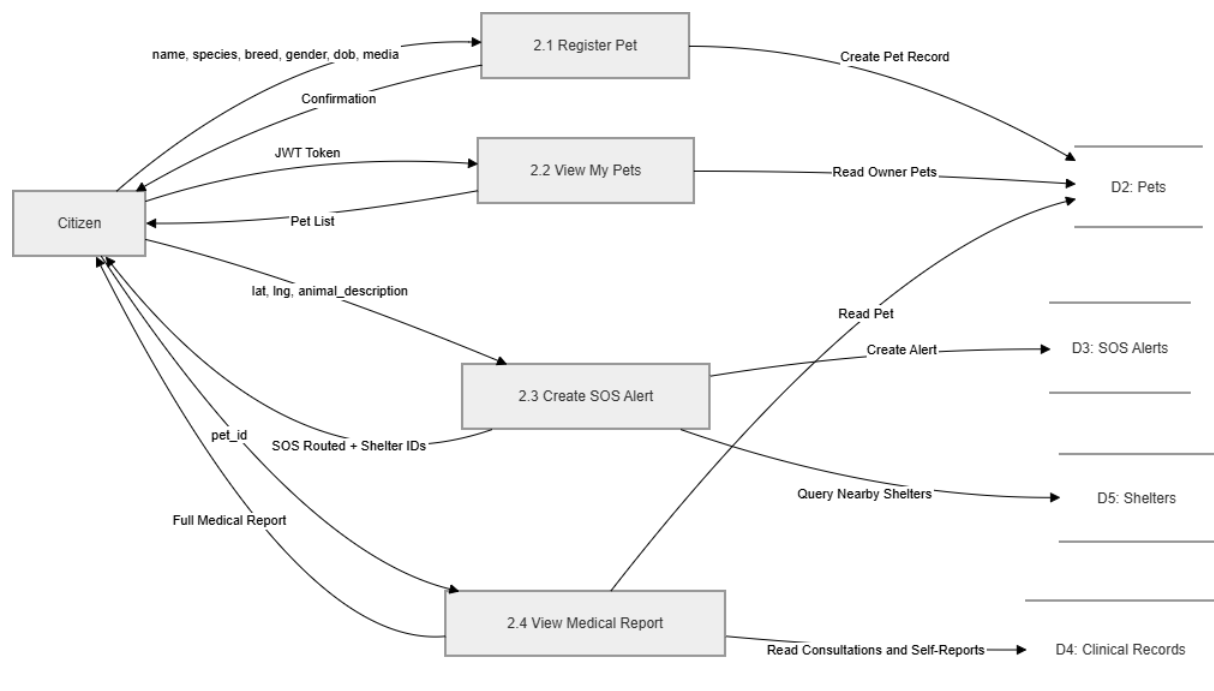
Level 0 — Context Diagram



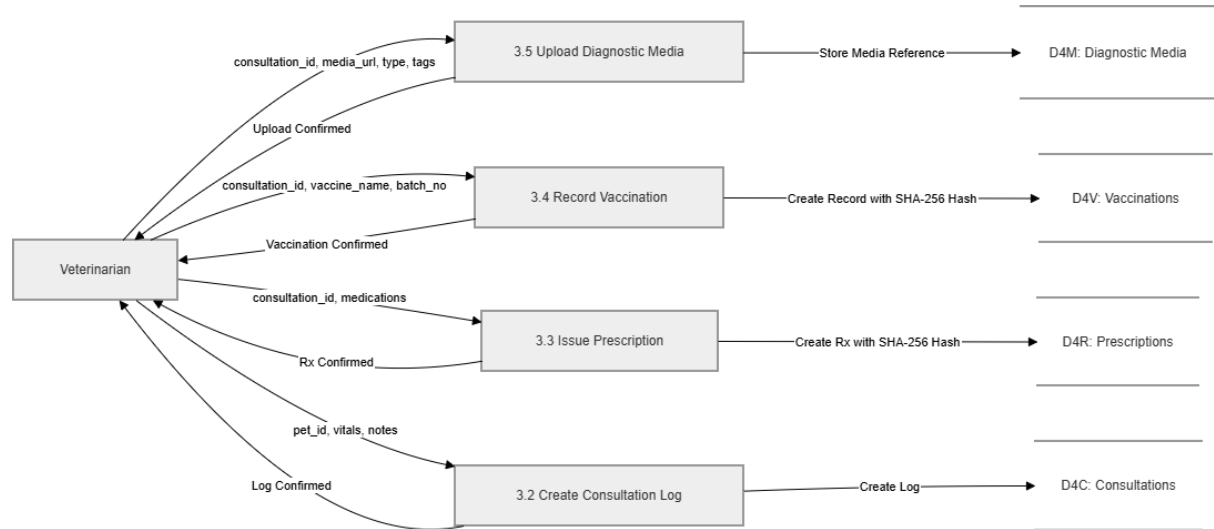
Level 1 — Module 1.0: Users & Authentication



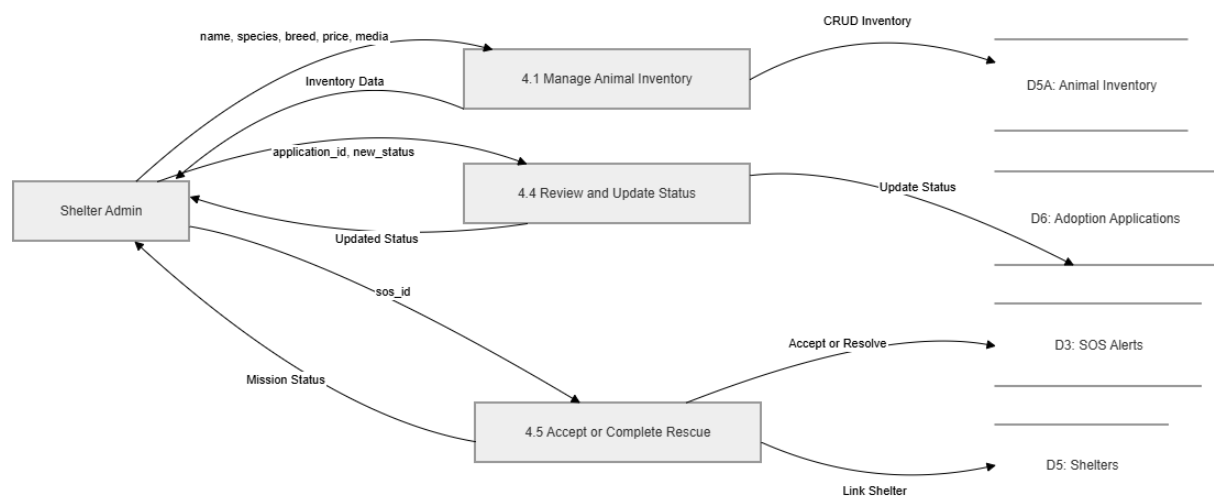
Level 1 — Module 2.0: Citizens



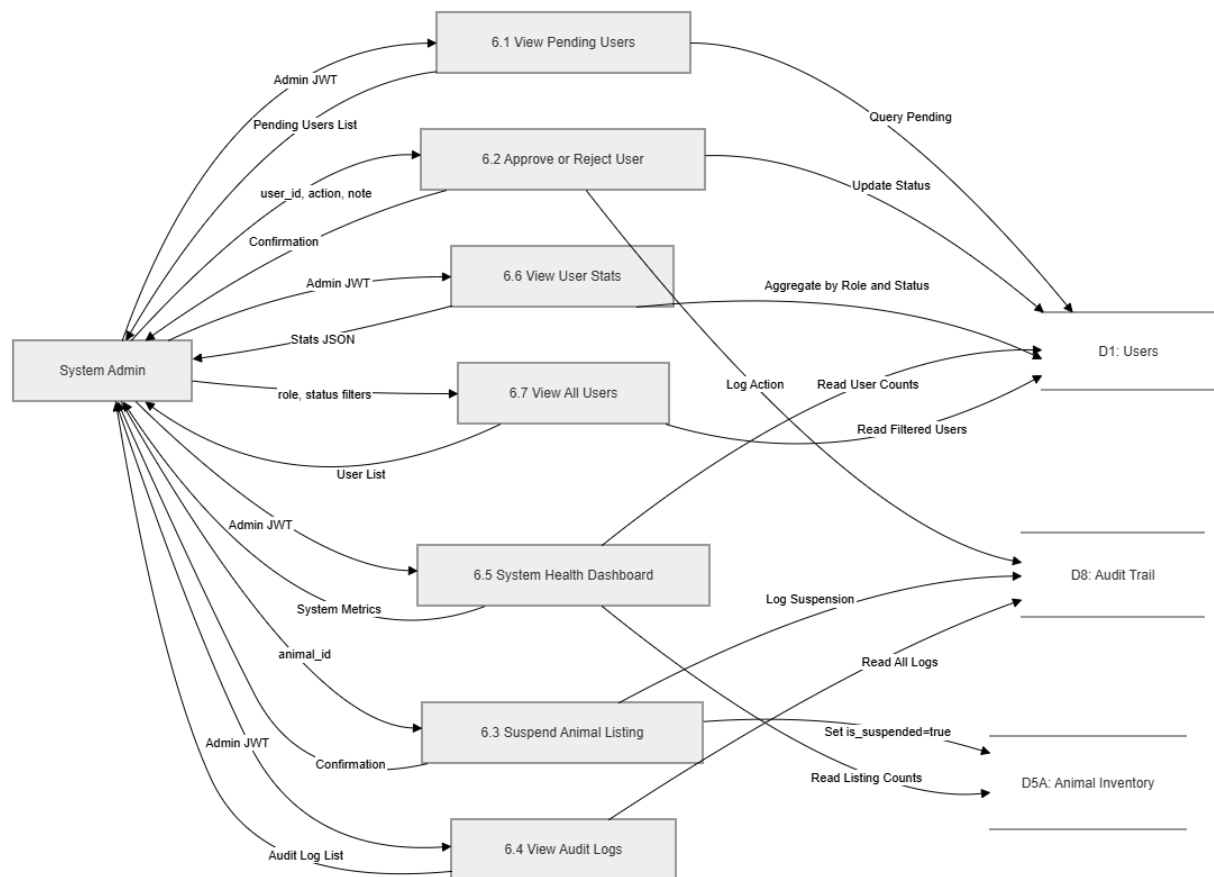
Level 1 — Module 3.0: Clinical



Level 1 — Module 4.0: Shelter & Adoption



Level 1 — Module 5.0: Governance (Super Admin)



Part D: Table Design

1. Identity & Access Control Tables

Table: Users_User (Custom Base User)

Field Name	Data Type	Constraint	Description
id	Integer	Primary Key, Auto-Increment	Unique identifier for every system user.
email	Varchar(255)	Unique, Not Null	Used as the login credential.
password	Varchar(128)	Not Null	Hashed password.
role	Varchar(50)	Not Null	Enum: Citizen, Vet, ShelterAdmin, Civic.
is_verified	Boolean	Default: False	True if SuperAdmin approved their ID.
date_joined	DateTime	Auto-Now-Add	Timestamp of registration.

Table: Users_VeterinarianProfile

Field Name	Data Type	Constraint	Description
id	Integer	Primary Key	Profile identifier.
user_id	Integer	Foreign Key (Users_User.id)	Links to base user account.
license_no	Varchar(100)	Unique, Not Null	Medical license for validation.

Field Name	Data Type	Constraint	Description
clinic_name	Varchar(200)	Nullable	Current place of practice.
specialization	Varchar(100)	Nullable	e.g. Surgeon, General, Avian.

2. Citizen & Public Action Tables

Table: Citizens_Pet

Field Name	Data Type	Constraint	Description
id	Integer	Primary Key	Unique Pet Passport ID.
owner_id	Integer	ForeignKey (Users_User.id)	The citizen who owns the pet.
name	Varchar(100)	Not Null	Pet's call name.
species	Varchar(50)	Not Null	e.g., Dog, Cat, Bird.
breed	Varchar(100)	Nullable	Specific genetic breed.
age_months	Integer	Nullable	Age representation.

Table: Citizens_SOSAlert

Field Name	Data Type	Constraint	Description
id	Integer	Primary Key	Alert ticket number.
reporter_id	Integer	ForeignKey (Users_User.id)	Citizen logging the alert.

Field Name	Data Type	Constraint	Description
latitude	Decimal(9,6)	Not Null	Geospatial location.
longitude	Decimal(9,6)	Not Null	Geospatial location.
status	Varchar(50)	Default: "Active"	Enum: Active, Rescued, Outbreak.
description	Text	Nullable	Detail of injury or missing animal.

3. Animal Shelter & Adoptions Tables

Table: Shelter_ShelterFacility

Field Name	Data Type	Constraint	Description
id	Integer	Primary Key	Facility identification.
name	Varchar(200)	Not Null	Name of the rescue shelter.
capacity	Integer	Not Null	Maximum allowed animals.
address	Text	Not Null	Physical location.

Table: Shelter_AnimalInventory

Field Name	Data Type	Constraint	Description
id	Integer	Primary Key	Internal tracking tag.
shelter_id	Integer	ForeignKey (ShelterFacility.id)	The facility holding the animal.
species	Varchar(50)	Not Null	Biological class.
health_status	Varchar(50)	Default: "Intake"	Status (e.g. Healthy, Sick, Quarantine).
is_adoptable	Boolean	Default: False	True if cleared by vet.

Table: Shelter_AdoptionApplication

Field Name	Data Type	Constraint	Description
id	Integer	Primary Key	Application form ID.
animal_id	Integer	ForeignKey (AnimalInventory.id)	Animal applied for.
applicant_id	Integer	Foreign Key (Users_User.id)	Citizen applying.
ai_score	Integer	Nullable	Generated compatibility score (0-100).
status	Varchar(50)	Default: "Pending"	Pending, Approved, Rejected.

4. Clinical & Health Tables

Table: Clinical_ConsultationLog

Field Name	Data Type	Constraint	Description
id	Integer	Primary Key	Record sheet ID.
pet_id	Integer	ForeignKey (Citizens_Pet.id)	Patient being checked.
vet_id	Integer	ForeignKey (Users_User.id)	Doctor conducting exam.
diagnosis	Varchar(200)	Not Null	Primary medical finding (for analytics).
datetime	DateTime	Auto-Now-Add	Time of clinical logging.

Table: Clinical_Vaccination

Field Name	Data Type	Constraint	Description
id	Integer	Primary Key	Immutable vaccine token.
pet_id	Integer	ForeignKey (Citizens_Pet.id)	Pet receiving vaccine.
vaccine_type	Varchar(100)	Not Null	e.g. Rabies, DHPP.
administered_by	Integer	ForeignKey (Users_User.id)	Vet who gave the shot.
expiry_date	Date	Not Null	Good-until date.

Table: Clinical_DiagnosticMedia

Field Name	Data Type	Constraint	Description
id	Integer	Primary Key	Media identifier.
consult_id	Integer	ForeignKey (ConsultationLog.id)	Link to the specific visit.
media_url	Varchar(500)	Not Null	AWS S3 URI string where file is hosted.
type	Varchar(50)	Not Null	e.g. X-Ray, Bloodwork PDF.